Initial Load
Lazy Loading &
Deferrable Views

Alexander Thalhammer | @LX_T

Outline - Initial Load Performance

- Assets & Build
- Lazy Loading & Deferrable Views
- SSR & SSG

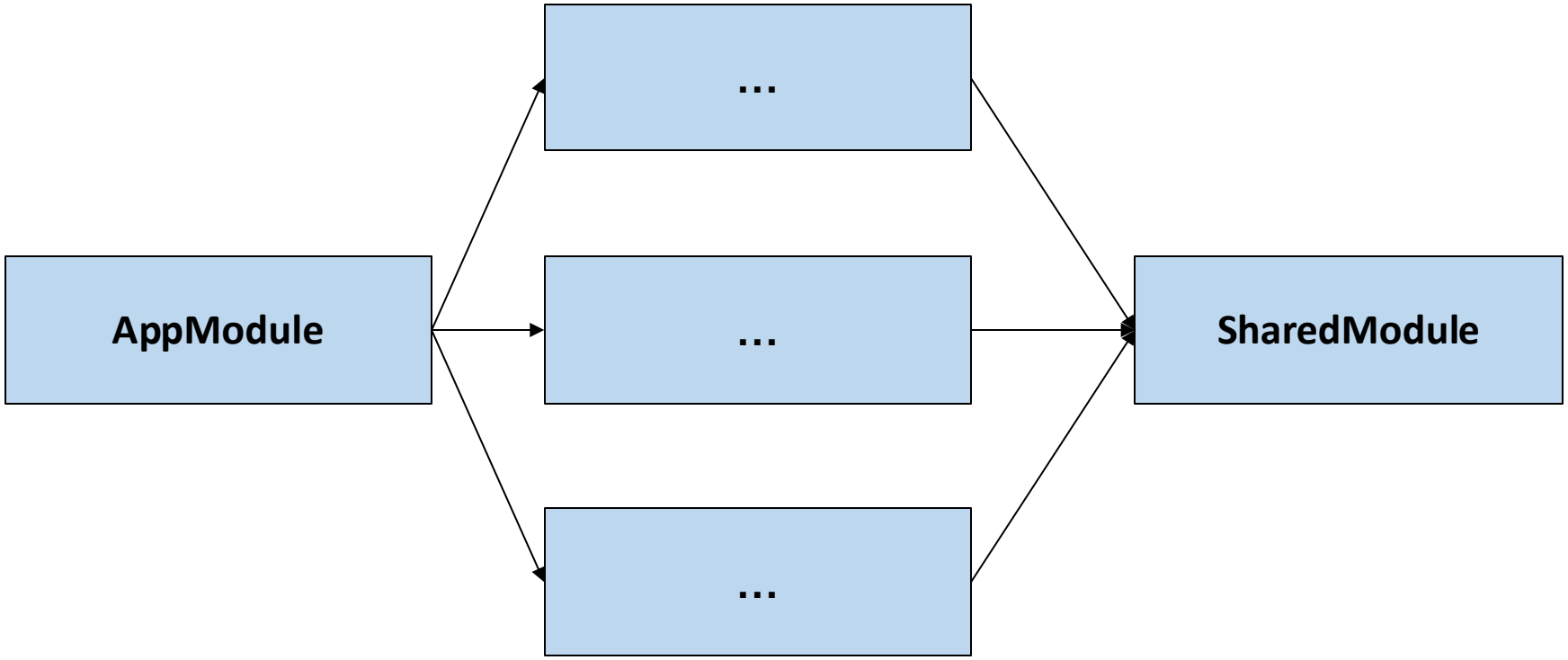
Lazy Loading

Lazy Loading & Deferring

- Lazy Loading via modules / features
  - 2 most common pitfalls and their solutions
- Lazy Loading via standalone components
- Preloading
  - PreloadAllModules
  - Other strategies
- ~~Lazy Loading without the router (a bit complicated)~~
- ~~Lazy Loading below the fold (very, very complicated)~~
- Deferring (brand new in NG 17, very lean, replaces last 2)

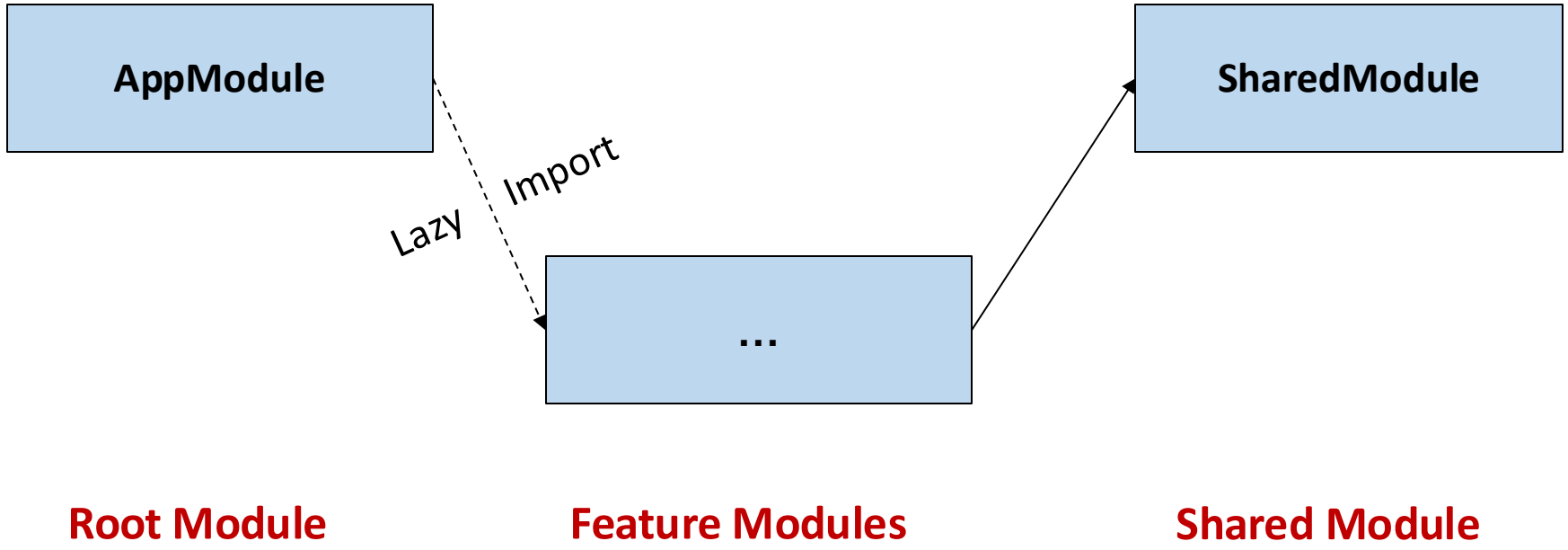
Agenda

Angular Module | Feature Structure

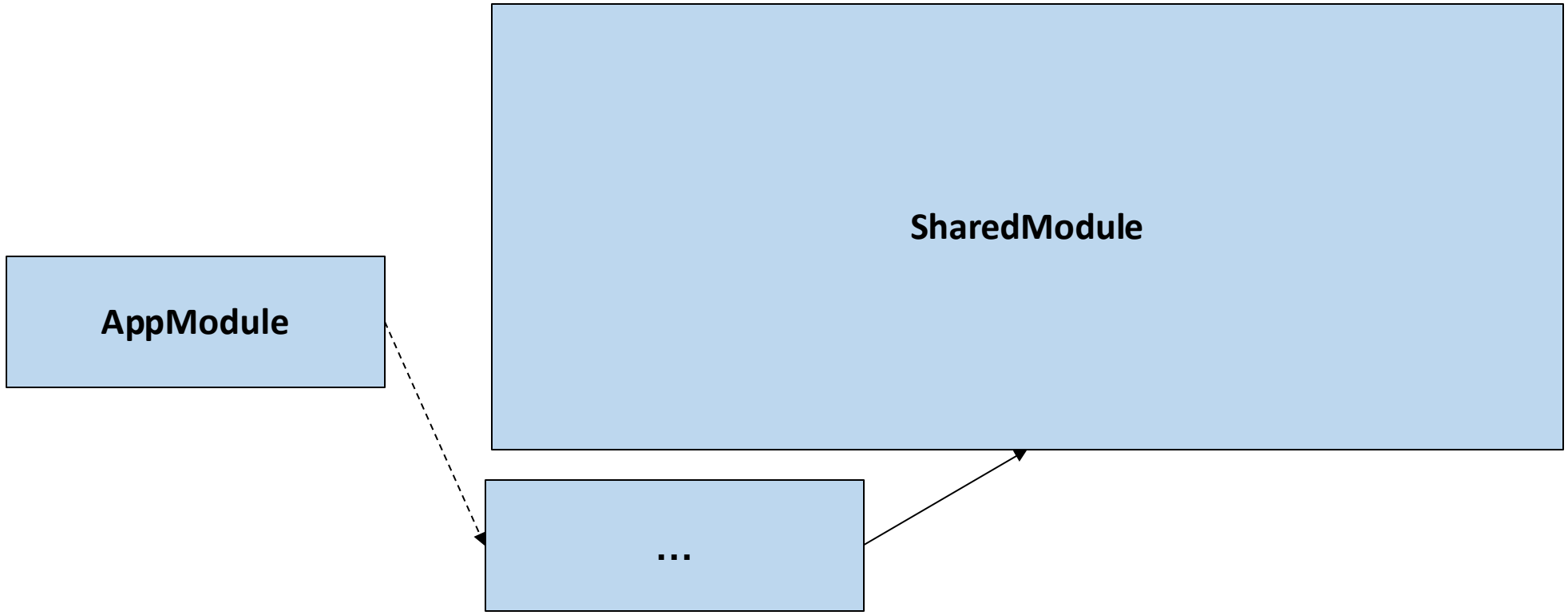


Root Module **Feature Modules** **Shared Module**

Angular Lazy Loading – Theory

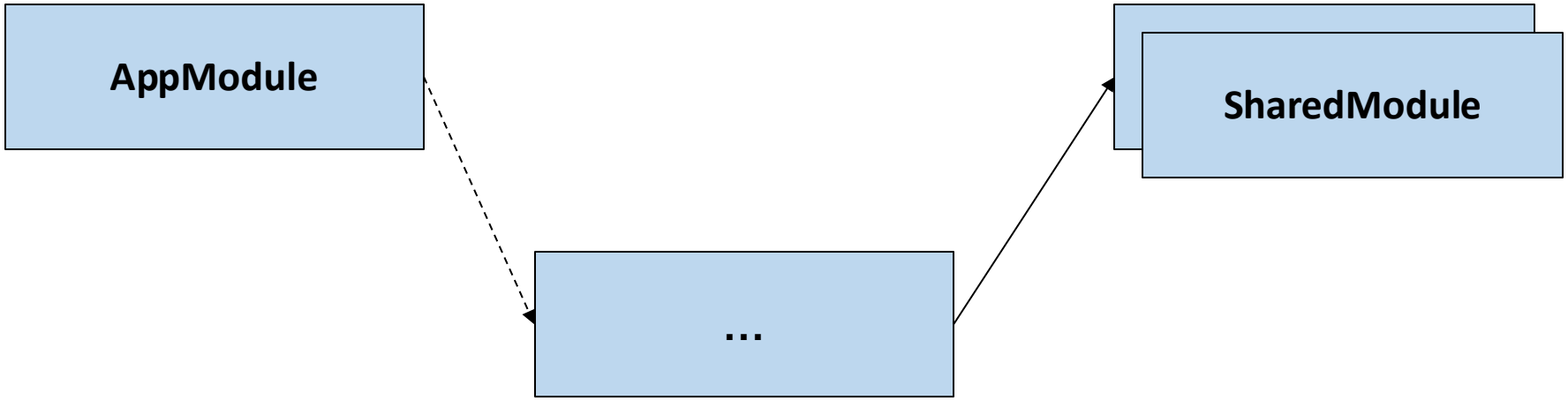


Angular Lazy Loading – Common Pitfall



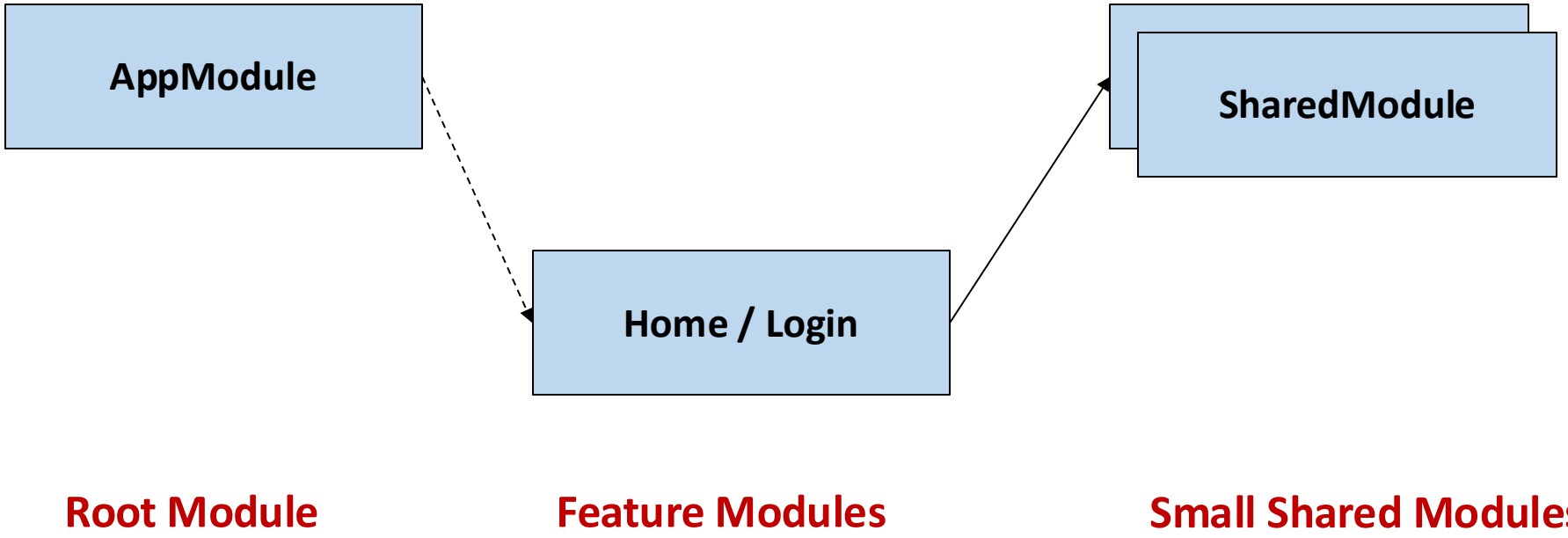
Root Module **Feature Modules** **Huge Shared Module**

Angular Lazy Loading - Solution

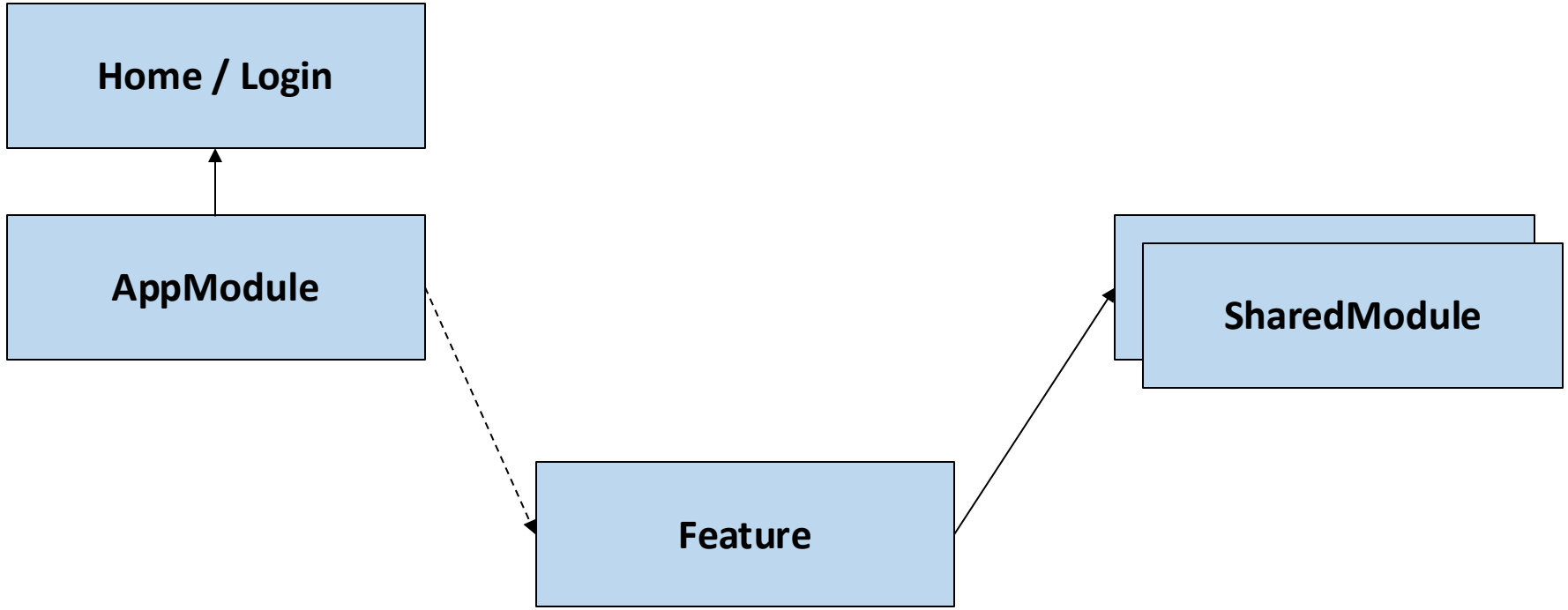


Root Module **Feature Modules** **Small Shared Modules**

Angular Lazy Loading – Another Pitfall



Angular Lazy Loading – Solution



Root Module **Feature Modules** **Small Shared Modules**

App Routes with Lazy Loading

```
export const appRoutes: Routes = [
    {
        path: 'home',
        component: HomeComponent
    },
    {
        path: 'flights_module',
        loadChildren: () => import('./flights/flights.module')
                             .then((m) => m.FlightsModule)
    },
    {
        path: 'flights_standalone',
        loadChildren: () => import('./flights/flights.routes')
    }
];
```

Routes for "lazy" Feature

```
export const flightsRoutes: Routes = [
    {
        path: 'flight-search',
        component: FlightSearchComponent,
        […]
    },
    […]
}

export default flightsRoutes; // for standalone
```

flights/ flight-search

Triggers Lazy Loading w/ loadChildren

Demo

Lazy Loading NG Module

ANGULAR
ARCHITECTS

Lazy Loading with standalone components

```
export const appRoutes: Routes = [
    {
        path: 'home',
        component: HomeComponent
    },
    {
        path: 'charts',
        loadComponent: () => import('./charts/charts.component')
                             .then((c) => c.ChartsComponent)
    }
];
```

Demo

Lazy Loading
Standalone Component

ANGULAR
ARCHITECTS

What about services?

```
…
@Injectable({
  providedIn: 'root'
})
…
```

- When used by 1 lazy loaded module/comp exlusively, it will be put into that chunk
- When used by 2 / more lazy loaded modules/comps, it will be put in a common chunk
- When used by an eagerly loaded part, it will be put into main bundle

Demo

Lazy Loading Services

ANGULAR
ARCHITECTS

Lazy Loading

- Lazy Loading means: Load it later, after startup
- Better initial load performance
- But: Delay during runtime for loading on demand

Preloading

Preloading

– Once the initial load (the important one) is complete load the lazy loaded modules (before they are even used)

– When module is needed it is available immediately

Activate Preloading (in AppModule)

```
…
imports: [
    […]
    RouterModule.forRoot(
        appRoutes, { preloadingStrategy: PreloadAllModules }
    );
]
…
```

Activate Preloading (in app.config.ts)

```
…
providers: [
    […]
    provideRouter(
        appRoutes, withPreloading(PreloadAllModules),
    ),
]
…
```

Demo

Lazy Loading
Preloading

ANGULAR
ARCHITECTS

Intelligent Preloading with ngx-quicklink

```
…
imports: [
    […]
    QuicklinkModule,
    RouterModule.forRoot(
        appRoutes, { preloadingStrategy: QuicklinkStrategy }
    );
]
…
```

https://web.dev/route-preloading-in-angular/

https://www.npmjs.com/package/ngx-quicklink

Demo

Lazy Loading
Quicklink

ANGULAR
ARCHITECTS

Or CustomPreloadingStrategy

```
…
imports: [
    […]
    RouterModule.forRoot(
        appRoutes, { preloadingStrategy: CustomPreloadingStrategy }
    );
]
…
```

Use Lazy Loading a lot - but carefully ;-)

- Solution: Implement lazy loading whereever you can
  - Use lazy loading with the router
    - Modules
    - Components (new since NG15!)
    - Maybe use a CustomPreloadingStrategy if App is very big
  - Use dynamic components
- Use Import Graph Visualizer to detect why things land in main
- But don't lazyload the initial feature, because it will be delayed ;-)
- Don't destroy lazy loading by (eagerly) loading huge shared module

Lazy Loading without the router

```
…
    @ViewChild('cnt', { read: ViewContainerRef }) vCR!: ViewContainerRef;
…
    async ngOnInit() {
        const esm = await import('./lazy/lazy.component');
        const lazyComponentRef = this.vCR.createComponent(esm.LazyComponent);
    }
…
```

```
…
    <ng-container #cnt></ng-container>
…
```

Demo

Lazy Loading
Dynamic

ANGULAR
ARCHITECTS

Critical Rendering Path / Above the fold

- Problem: Bad PageSpeed Score that cannot be fixed with #1
- Identify: Initial load is too slow
  - Using Lighthouse / PageSpeed Insights or
  - WebPageTest
- Solution: Use custom lazy loading of content below the fold
  - Not trivial
  - Has to be implemented manually

Deferrable Views

ANGULAR
ARCHITECTS

Deferrable Views

– Problem: Lazy Loading without the router and especially Lazy Loading below the fold is rather complicated and inconvenient

– NG17 has the solution in the new control flow template syntax

– It's called: **Deferrable Views**

Deferrable Views - syntax

```
…
@defer (on viewport) {
  <aa-lazy-component />
} @placeholder {
  <p>Component is loading on viewport.</p>
}
…
```

Deferrable Views - on

- on immediate (default)
- on viewport
- on hover
- on interaction
- on timer(4200ms)

Deferrable Views - when

- specifies an imperative condition as an expression that returns a bool
  - best used: boolean flag

- if the condition returns to false, the swap is not reverted
  - it is a one-time operation

```
…
@defer (when condition) {
  <aa-lazy-component />
}
…
```

Deferrable Views - prefetch

– allows to specify conditions **when prefetching** of the dependencies should be triggered

```
…
@defer (on viewport; prefetch on idle) {
 <aa-lazy-component />
}
 …
```

Deferrable Views - extras

- @placeholder
- @loading
- @error

```
…
@defer (on viewport; prefetch on idle) {
  <aa-lazy-component />
} @placeholder (minimum 500ms) {
  <img width="420" height="420" alt="lazy component placeholder"
src="placeholder.avif" />
} @loading (after 500ms; minimum 1s) {
  <img width="420" height="420" alt="lazy is loading spinner"
src="spinner.avif" />
} @error {
  <p>Why do I exist?</p>
}
…
```

Demo

Lazy Loading
Deferrable Views

ANGULAR
ARCHITECTS

Lab 04 Angular Lazy Loading

Module / Standalone Component / Deferrable View

ANGULAR
ARCHITECTS

Lazy Loading & Deferring

- **Lazy Loading via modules / features**
  - 2 most common pitfalls and their solutions
- **Lazy Loading via standalone components**
- **Preloading**
  - PreloadAllModules
  - or QuicklinkStrategy with ngx-quicklink
- **Deferring** (brand new in NG 17, very lean)

Recap

References

- Angular Docs
  - Lazy-loading feature modules
- Angular Architects Blog
  - Deferrable Views (Blog post)

Questions?

ANGULAR
ARCHITECTS